

Towards Measuring Vulnerabilities and Exposures in Open-Source Packages

Tobias Dam, Sebastian Neumaier



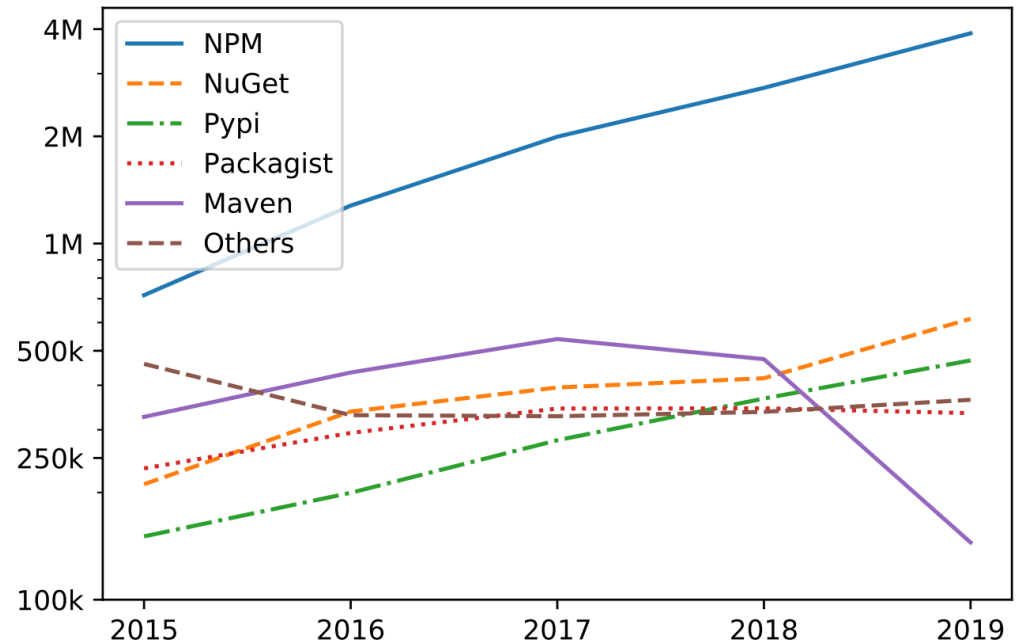
What is Open Source?

Open-source software is software with publicly available code. This means that **anyone can not only inspect the source code but also modify it** and enhance it.

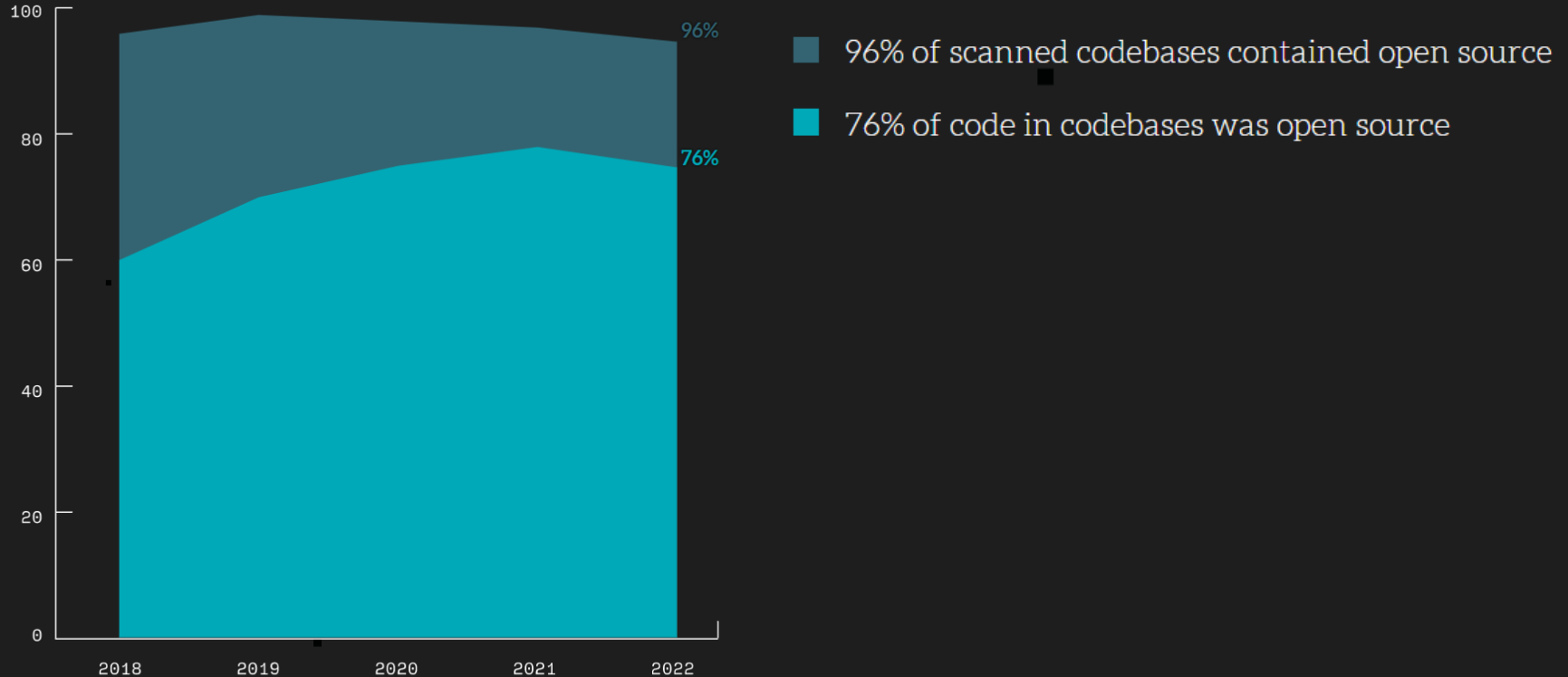


The Open Source Packages Landscape

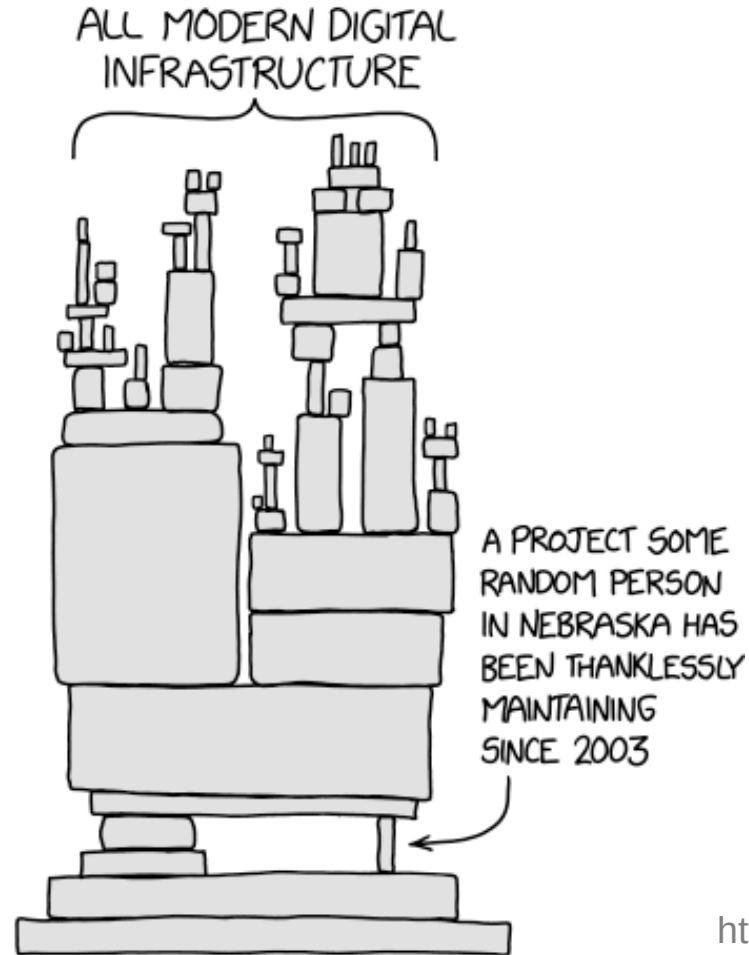
Project Manager	Projects	Perc.
Go	1,818,666	39.45
NPM	1,275,082	27.66
Packagist	313,278	6.80
Pypi	231,690	5.03
NuGet	199,447	4.33
Maven	184,871	4.01
Ruby	161,608	3.51
Others	425,020	9.22



Open Source is everywhere



Dependencies...



For instance: Heartbleed (2014)

- Vulnerability in OpenSSL
- Large proportion of online services, websites, VoIP phones, routers, etc. were vulnerable
- Was implemented by the only permanent developer

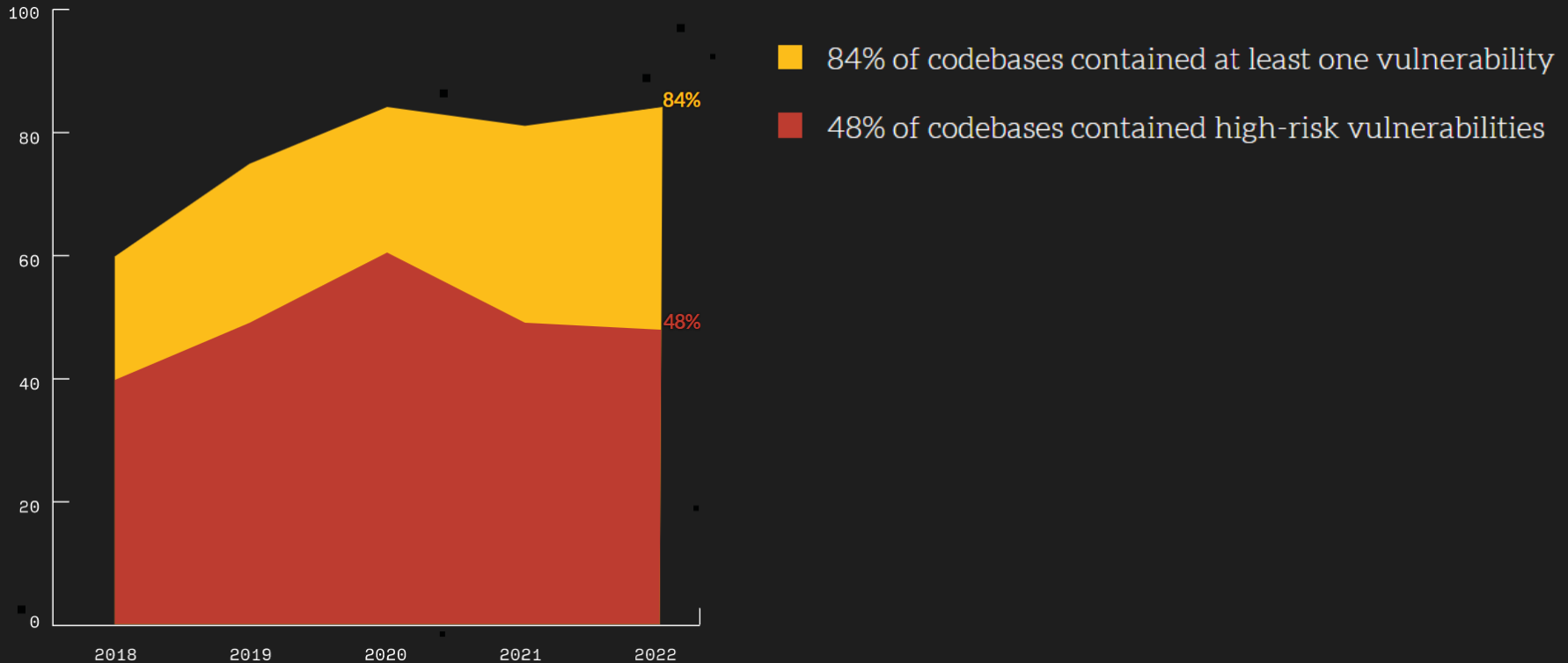
SERVER, ARE YOU STILL THERE?
IF SO, REPLY "HAT" (500 LETTERS).



a connection. Jake requested pictures of users.
User Meg wants these 500 letters: HAT. Lucas
requests the "missed connections" page. Eve
(administrator) wants to set server's master
key to "14835038534". Isabel wants pages about
snakes but not too long". User Karen wants to
change account password to "CoHeRaSt". User



Security Risk is prevalent



Common Vulnerability and Exposures (CVE)

CVE-Search / CVE-2021-44631

ID	CVE-2021-44631
Summary	A Buffer Overflow vulnerability exists in TP-LINK WR-886N 20190826 2.3.8 in the /cloud_config/router_post/reset_cloud_pwd feature, which allows malicious users to execute arbitrary code on the system via a crafted post request.
References	• https://github.com/Yu3H0/loT_CVE/tree/main/886N/resetCloudPwdRegister ▼
Vulnerable Configurations	• cpe:2.3:o:tp-link:tl-wr886n_firmware:20190826_2.3.8:*:*:*:*:* ▼ • cpe:2.3:h:tp-link:tl-wr886n:-:*:*:*:*:*
CVSS	Base: 10.0 (as of 12-03-2022 - 04:23) Impact: Exploitability:
CWE	CWE-120

2015	2016	2017	2018	2019	2020	2020
6,602	6,520	18,161	18,213	19,095	20,516	22,226

Research Goal

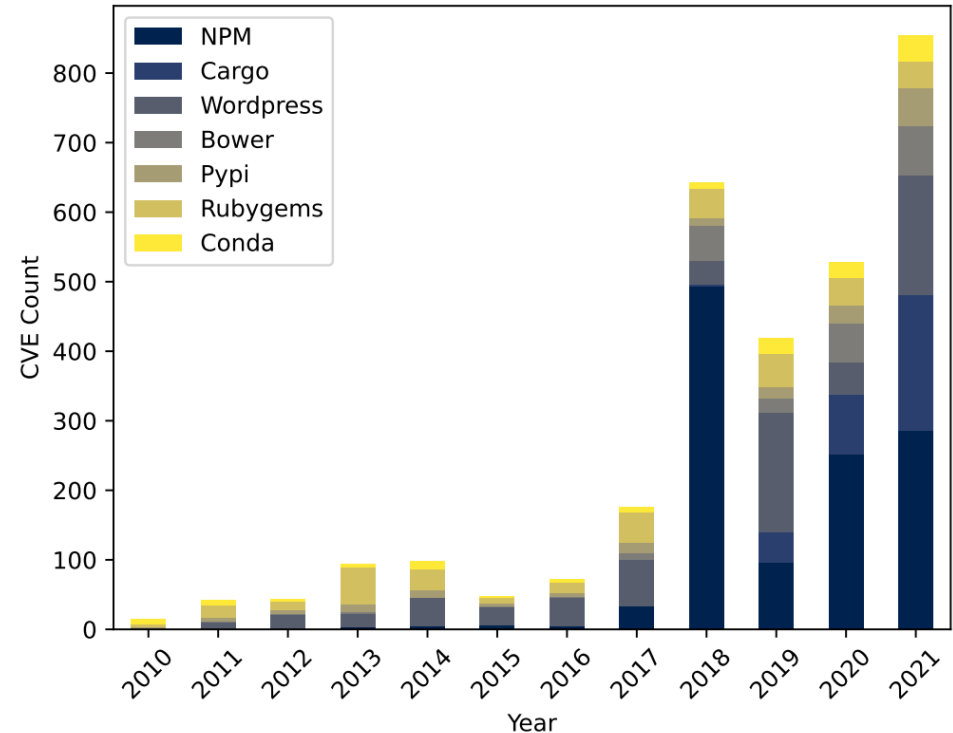
- Find corresponding Software repositories, packages, and CVE entries
 - Mapping of CVE entries to open-source projects
 - Quantitative analysis of the frequency and distribution of CVE
 - Identify challenges and quality issues wrt. the mapping

Mapping Approaches

- Open Data dump of the Libraries.io database
 - Includes metadata such as package manager, repository (e.g., GitHub) links, owner and keywords
- Evaluation of different CVE mapping approaches
 - Based on matching repository names, fuzzy string matching, and URLs

Findings

- Insights regarding number of vulnerabilities for certain package managers
- Noticeable upwards trend in vulnerabilities in recent years
- Improved approach evaluation is necessary



Identified Mapping Challenges

1. CVE entries do not contain clear identifiers of the code base
2. Incomplete CVE entries and potentially wrong attribute values
3. Multiple relevant packages to the CVE entry
4. No complete and reliable index of open source projects

Is this information sufficient..?

Are the reported bugs and vulnerabilities sufficient to make statements about the health of open source software projects?

Operational Risks

no development activity, no user updates, no feature upgrades or code improvements



89% were more than 4 years out of date

91% contained components that weren't the current version

91% had received no development activity in the last 2 years

88% contained components with no activity in the last 2 years and contained components that weren't the latest version

Towards a Critical Open-Source Software Database

Tobias Dam, Lukas Daniel Klausner, and Sebastian Neumaier.

To appear in WWW'23, Companion Proceedings of the ACM Web Conference 2023.

Goal

- Assess the "health" of (especially: critical) OSS projects
- "Health" is to be defined by various metrics
 - Security, stability, resilience, compliance, etc.
- Comprehensive analysis and ongoing evaluation on large software repositories

Quantitative & qualitative metrics are to be applied

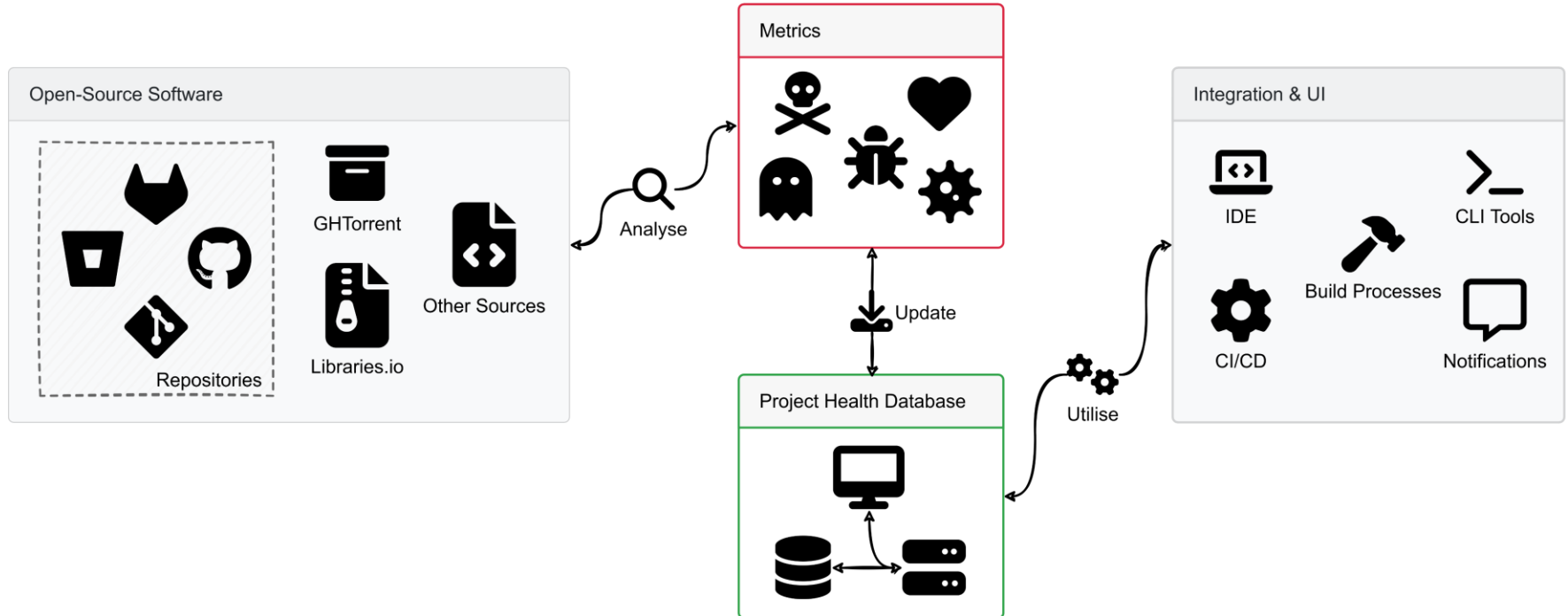
- *Quantitative & automated:*
Crawling of meta-information (e.g. contributors, commits, activity),
analysis of dependencies
- *Qualitative:*
sustainability/funding, activity indicators, security policies

Metrics (Cont'd)

Categories

- *Relevance metrics*: such as retrieval numbers, map the popularity and visibility of projects
- *Activity metrics*: highlight critical points, e.g. development failure or incipient inactivity
- *Security metrics*: e.g. known CVE vulnerabilities can show security issues

Platform Architecture



Next Steps

- Database of continuous assessment of metrics
 - Relevance and criticality become visible
 - Identify which support is target-oriented, efficient and effective (financially, working time, know-how, ...)
- Web platform displaying the results
 - Support for developers in software selection and continuous safety checks
 - Basis for making informed decisions about funding and support

Sebastian Neumaier

St. Pölten University of Applied Sciences
Austria

sebastian.neumaier@fhstp.ac.at

<https://sebneumaier.wordpress.com/>

<https://twitter.com/sebneum>

